# Midterm 2: Solving a 4x4 Maze

Published | Assign To | Edit

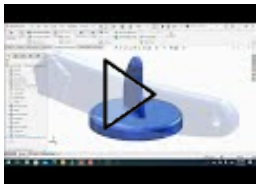
The goal of this project is to develop a solution to solve 4x4 rectangular mazes with the. The robot's end effector must navigate through the maze in straight-line sections. A maze will be printed on a plastic board placed within the AI kit's camera zone. The algorithm to solve the maze must automatically determine the entrance, solution path and the exit of the maze. The input to the maze solution algorithm will be an image taken by the AI Kit's camera. The solution path will be consequent straight lines that start at the maze entrance and ends at the maze exit. The solution algorithm should work for any 4x4 maze generated via the **maze generation tool** (https://www.mazegenerator.net/) . The robot end effector must autonomously navigate the solution path.

## Literature References

(https://github.com/Robotics-and-Dynamical-Systems-Lab/RAS545/blob/main/Final_Project/instructions.md#literature-references)

Go through all the literature references. They'll be helpful in developing the project.

- MATLAB TCP/IP Communication: **Learn More** (https://www.mathworks.com/help/matlab/matlab-engine/tcp-ip-communication.html)
- ArUco Markers: **Overview of ArUco Markers** (https://www.vision.ee.ethz.ch/en/vision/software/aruco.html)
- OpenCV for ArUco Detection: **OpenCV Documentation** (https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html)
- Learning Orbis's Robotics playlist for modeling robots in MATLAB. See **here** (https://www.youtube.com/playlist?list=PLWF9TXck7O_ymYWT8Q33omPb5K-A5v4Ae)
- Simcape Multibody simulation in MATLAB. See **here** (https://www.youtube.com/watch?v=pDiwAA1cnb0&t=2283s)



(https://www.youtube.com/watch?v=pDiwAA1cnb0&t=2283s)

- Robot forward kinematics in MATLAB. See **here** (https://www.youtube.com/watch?v=xpA8TKEMpMk)



(https://www.youtube.com/watch?v=xpA8TKEMpMk)

- Robot trajectory planning in MATLAB. See **here** (https://www.youtube.com/watch?v=Fd7wjZDoh7g)



(https://www.youtube.com/watch?v=Fd7wjZDoh7g)

- What are URDFs. See here (https://www.youtube.com/watch?v=CwdbsvcpOHM)



(https://www.youtube.com/watch?v=CwdbsvcpOHM)

- Working with robots in ROS (basics). See here (https://www.youtube.com/watch?v=2llV3dRvHmQ&list=PLunhqkrRNRhYYCaSTVP-qJnyUPkTxJnBt)



(https://www.youtube.com/watch?v=2llV3dRvHmQ&list=PLunhqkrRNRhYYCaSTVP-qJnyUPkTxJnBt)

- Simulating robot kinematics in ROS. See here (https://www.youtube.com/playlist?list=PLcGRzrc4DaVyt1RLw40FRTNIvHhwIlSNA)
- IKfast Module in ROS is useful to solve inverse kinematics and customize your solution. See here (https://openrave.org/docs/latest_stable/openravepy/ikfast/#ik-types)
- Creating an IK Fast solution in ROS. See here (https://wiki.ros.org/Industrial/Tutorials/Create_a_Fast_IK_Solution)
- An approach to solve mazes in MATLAB. See here (https://www.mathworks.com/matlabcentral/answers/711803-random-maze-solving-algorithm-with-matrix) . Watershed Transform method (https://www.mathworks.com/company/technical-articles/tips-and-tricks-solving-a-maze-with-the-watershed-transform.html) . BFS based maze solver (https://github.com/Mouad4399/-Maze-Solver-In-Matlab-Using-DFS-BFS) .
- An approach to solve mazes in Python. See here (https://thecleverprogrammer.com/2021/01/26/maze-solver-with-python/) . Recursivity and A* search see here (https://www.laurentluce.com/posts/solving-mazes-using-python-simple-recursivity-and-a-search/)

# Project Details

## Mid_Term_2_Starter_Code.zip (https://canvas.asu.edu/courses/237283/files/122820104?wrap=1)

# (https://canvas.asu.edu/courses/237283/files/122820104/download?download_frd=1)

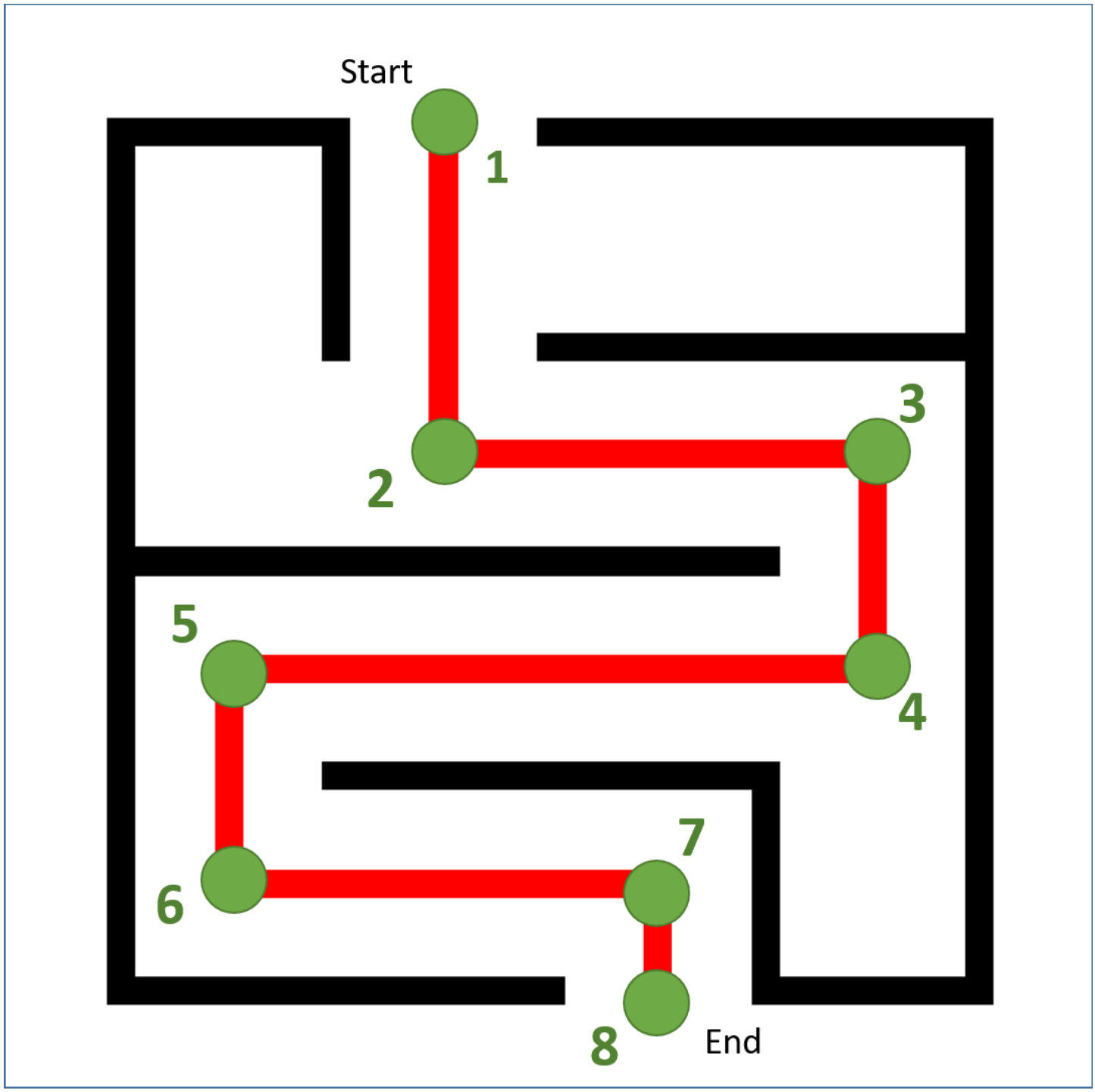
(https://github.com/Robotics-and-Dynamical-Systems-Lab/RAS545/blob/main/Final_Project/instructions.md#project-details)

## 1: Maze Generation and solving the maze

(https://github.com/Robotics-and-Dynamical-Systems-Lab/RAS545/blob/main/Final_Project/instructions.md#1-maze-generation-and-solving-the-maze)

1a. **Learn about generating a 4x4 Maze**: Use the **maze generation tool** (https://www.mazegenerator.net/) to create a 4x4 maze. (See **sample maze** (https://github.com/Robotics-and-Dynamical-Systems-Lab/RAS545/tree/main/Final_Project/sample_mazes) )

- The maze should have a width and height of 4 cells each.
- Inner width and inner height of the maze will be 0.
- The maze can either start at the top or at the bottom **but NOT at the inner room**
- The E and R values control the maze generation algorithm. They can be anywhere between 0 and 100.
- Scale and position the maze within the robot's workspace of **3.5x3.5 inches**.
- View the **help** (https://www.mazegenerator.net/Help.aspx) about the maze generator tool.
- Below is a sample image of a randomly generated 4x4 maze starting from the top.




- 1b. **Solving the maze**: Students must determine an algorithm to solve the 4x4 maze
  - The algorithm to solve the maze must produce the physical waypoints for navigating the solution path from its image. Hence a physical interpretation of the robot's workspace must be obtained from pixel values of the solution path.
  - The image below is a representation of the top view of a possible setup where the starting point is close top the robot and the end point is close to the camera.

# Demonstration Judgement Criteria

- End effector must always maintain a safe distance vertical distance from the plastic board. (ideally between 10 to 20 centimeters).

- End effector should follow a path as straight as possible, and should be similar to the expected solution path.
- Your ingenuity of the maze solution approach is very important. There are several ways to solve the maze. Your code should work for all 4x4 rectangular mazes.
- In addition to the above points, students will be judged on the basis of how efficiently they solve the problem.
- The reliability and robustness of the solution also matters especially when determining physical solutions by processing the image of the maze.
- It is not necessary, but you should build fail-safes into your solution. For example: you could use a the escape key to make the robot return to a safe home position.

| | |
|---|---|
| Points | 40 |
| Submitting | a file upload |

| Due | For | Available from | Until |
|---|---|---|---|
| Nov 7, 2025 | Everyone | - | - |

+ **Rubric**